

Integración de Sistemas de Información

Práctica de Integración Hardware, Software, Firmware (HSF)

Presentado por:

Enrique Josué Molina Cárdenas

Código: 2343906

Docente:

Crisman Martínez Barrera

Universidad Santo Tomás

Decanatura de División de Educación Abierta y a Distancia

Ingeniería en Informática

Centro de Atención Universitario:

Bogotá

2026

1. Introducción

El desarrollo de esta actividad está centrado en exponer lo aprendido durante el segundo corte del espacio académico de Integración de Sistemas de Información, para lo cual se desarrolló una actividad práctica que consiste en integrar las tres capas fundamentales de los sistemas informáticos, las cuales son el hardware, el firmware y el software mediante el diseño, implementación y prueba de una aplicación en Python que interactúa con recursos físicos de forma real o simulada.

2. Caso de Uso

Para el desarrollo particular presentado en este documento, se realizó un informe respecto a la construcción realizada de un prototipo de monitoreo de un dispositivo industrial simulado. El dispositivo publica periódicamente temperatura, vibración y voltaje. La aplicación identifica la fuente, valida la telemetría, clasifica el estado, consulta recursos del sistema operativo y genera evidencias para la toma de decisiones. Debido a que no se cuenta con el hardware requerido para hacer la actividad se codificó con Python una simulación del firmware y usando el computador como hardware anfitrión.

3. Análisis del Problema

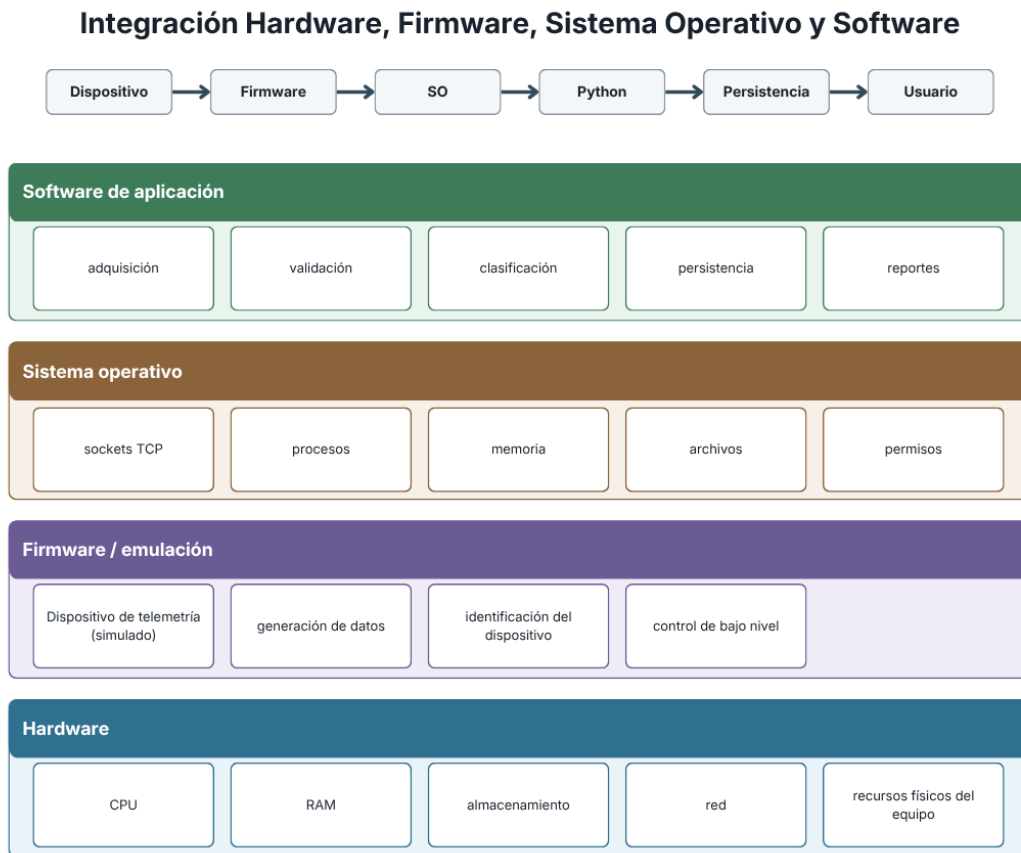
En base al caso de uso documentado, se plantean:

- Actores del sistema: dispositivo/emulador, aplicación Python, sistema operativo y usuario técnico.
- Entradas: tramas JSON delimitadas por salto de línea.
- Salida: CSV, JSON de resumen, log y estado de integración.
- Criterios de aceptación: al menos 95% de comunicación, datos válidos medibles, anomalía detectable y recuperación ante errores sin cierre inesperado.

4. Diseño de Arquitectura Integrando Firmware y Sistema Operativo

En base a la arquitectura planteada en la ruta de aprendizaje, se diseñó el sistema mediante los diagramas de capas, arquitectura, flujo de datos y secuencia, los cuales representan la misma idea desde perspectivas distintas, pero todas apuntan a mostrar cómo la telemetría pasa de una fuente física o simulada hasta la toma de decisiones:

Figura 1. Diagrama de capas del proyecto.

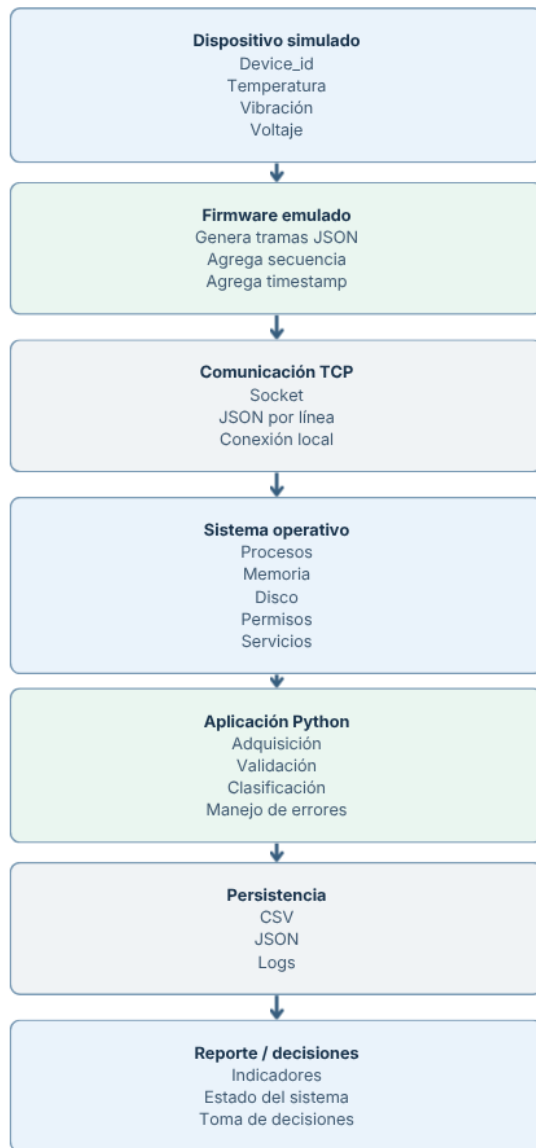


Nota. Hecho en Canva.

El diagrama de capas organiza el sistema en niveles: hardware, firmware, sistema operativo y software. Aquí se evidencia que el hardware genera o captura la señal, el firmware emula o controla la fuente de datos, el sistema operativo ofrece servicios como sockets, procesos, memoria, archivos y permisos, y el software de aplicación en Python realiza la adquisición, validación, clasificación, persistencia y generación de reportes. En resumen, este esquema muestra la responsabilidad de cada nivel. Posteriormente, se presenta el diagrama de arquitectura:

Figura 2. Diagrama de arquitectura del proyecto.

Diagrama de arquitectura

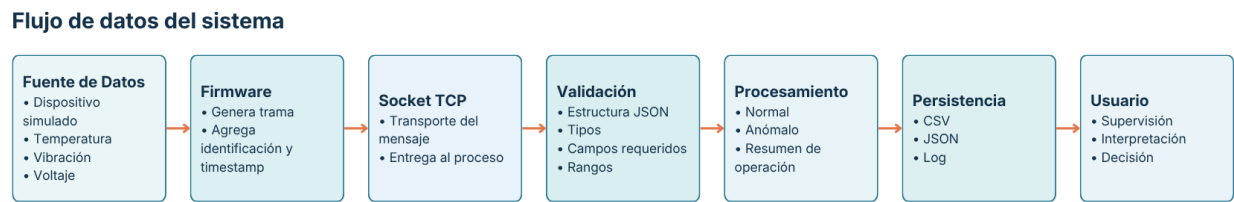


Nota. Hecho en Canva.

Este diagrama muestra los componentes específicos, de modo que se observa: un dispositivo simulado con id, temperatura, vibración y voltaje; un firmware emulado que arma tramas JSON; una capa de comunicación TCP; el sistema operativo como mediador; la aplicación Python como núcleo de análisis; y la persistencia en CSV, JSON y logs.

Diagrama de flujo de datos:

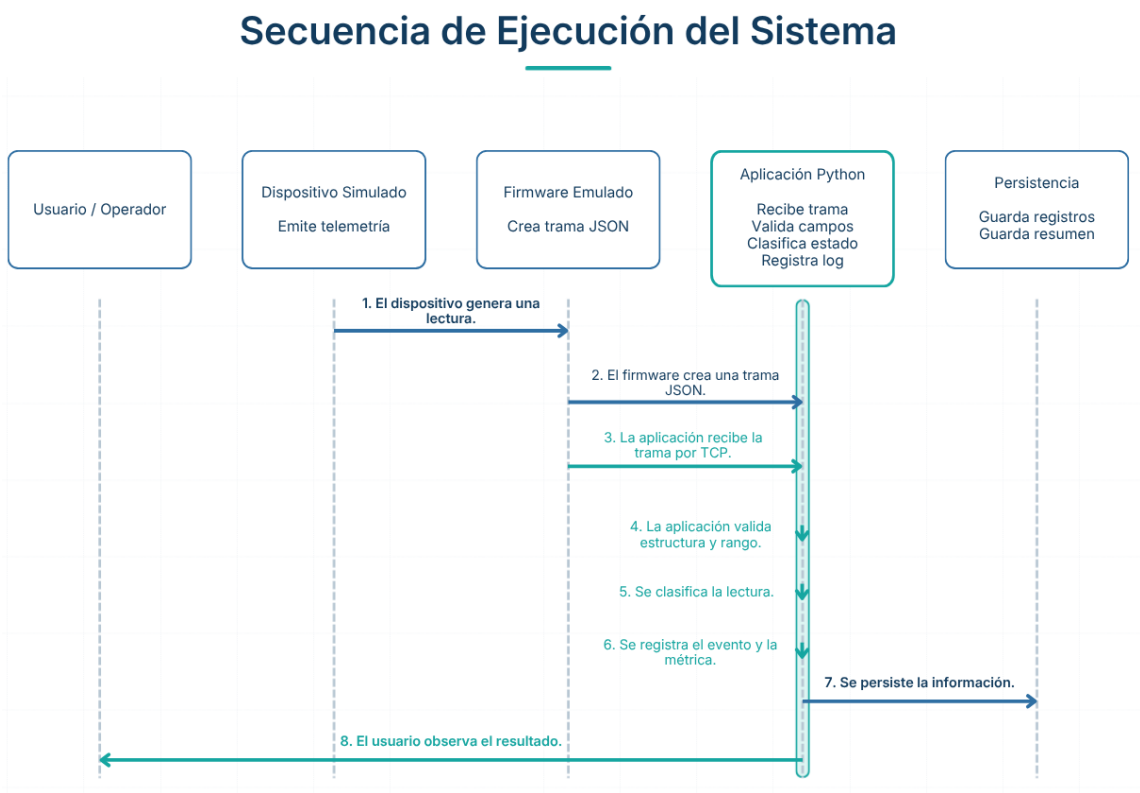
Figura 3. Diagrama de flujo de datos del proyecto.



Nota. Hecho en Canva.

El flujo de datos describe el recorrido real del dato: desde la adquisición hasta la validación, clasificación y almacenamiento. Posteriormente se tiene un diagrama de secuencias que muestra el orden en que ocurren los eventos: el firmware envía la trama, la aplicación la recibe, valida estructura y rangos, clasifica el estado, guarda la evidencia y permite al usuario interpretar la condición. En conjunto, estos diagramas muestran que el sistema no solo “recoge datos”, sino que los transforma en información para la toma de decisiones.

Figura 4. Diagrama de secuencia de ejecución del proyecto.



Nota. Hecho en Canva.

En general la arquitectura consiste primero en un hardware real, el cual es la CPU, memoria, reloj, almacenamiento y tarjeta de red del computador en el que se ejecute la aplicación de Python (en un despliegue real también sería el sensor y la placa); su responsabilidad es producir o transportar señales físicas. El firmware está representado por 'TelemetryDevice': asigna el identificador, toma muestras simuladas, crea la trama y decide cuándo enviarla, y no realiza el reporte ni la persistencia.

Por otra parte, el sistema operativo proporciona proceso, memoria, socket TCP, sistema de archivos, permisos y acceso a información de plataforma. Python implementa adquisición, validación, clasificación, persistencia y presentación textual. La transformación de valores aleatorios a una trama ocurre en el firmware emulado de la siguiente forma: de bytes a objetos ocurre en adquisición; de mediciones a los estados 'NORMAL' O 'ANÓMALO' ocurre en procesamiento; de objetos a CSV/JSON ocurre en la persistencia.

La interfaz es TCP local ('127.0.0.1') y JSON Lines. Cada mensaje contiene los atributos 'device_id', 'sequence', 'sent_at', 'temperature_c', 'vibration_g' y 'voltage_v'. TCP entrega orden y control de conexión; JSON facilita inspección de los datos enviados, así como su autenticación y un esquema versionado.

5. Implementación en Python

Como se puede observar en el repositorio en GitHub (principalmente en la carpeta 'src'), y en base a la arquitectura planteada, se observa que los módulos separan responsabilidades: 'simulator.py' emula firmware; 'models.py' se encarga de hacer validaciones respecto a la estructura de los datos, tipos, fecha y rangos; 'acquisition.py' controla socket y excepciones; 'processing.py' clasifica los datos según los datos planteados y genera resúmenes; 'system_info.py' consulta datos del Sistema Operativo (el cual este integra mediante el socket TCP, os.getpid, platform, shutil.disk_usage, y psutil si está instalado para memoria y cantidad de procesos); 'persistence.py' genera CSV, JSON y logs; 'cli.py' conecta la demo. La lectura de datos se realiza de forma periódica y cada trama se procesa de forma independiente.

Respecto a la entrada de datos, estos se decodifican usando el sistema UTF-8, y después se verifica que los datos vengan en formato JSON, que no haya campos vacíos, y se valida de forma particular: que la fecha esté en formato AAAA-MM-DD, que la temperatura este en un

rango de -40°C hasta 125°C, que la vibración esté en un rango de 0g a 10g, y el voltaje de 0V a 24V. Cuando se detecta un dato inválido este se rechaza, registra y no detiene las demás tramas. Se registran fecha/hora, nivel, evento, dispositivo, secuencia y resultado. Por otra parte, si el SO revoca permisos, se produce un error ‘persistence_error’ en el log; el proceso conserva el resumen en memoria y termina controladamente. También se controlan los errores de pérdida de comunicación o ausencia de hardware.

6. Pruebas y Fallos Controlados

Se realizan pruebas respecto al flujo normal del programa, como la identificación de errores, anomalías, JSON corrupto, rangos inválidos, desconexión, dispositivo ausente y persistencia. La ejecución de las pruebas a través de una terminal se puede realizar ejecutando el comando ‘py -m pytest -q’. En base a esto se presenta la siguiente matriz de pruebas:

Figura 5. Matriz de pruebas del proyecto.

ID	Prueba	Entrada/condicion	Resultado esperado	Resultado real	Estado
TP-01	Recepcion normal	3 tramas JSON validas	3 aceptadas y CSV generado	3 aceptadas	PASS
TP-02	Identificacion	<code>device_id=SIM-001</code>	ID conservado en registros/reporte	Conservado	PASS
TP-03	Anomalia	Temperatura 95 C	Clasificar ANOMALO sin detenerse	Clasificada	PASS
TP-04	JSON corrupto	<code>{bad-json}</code>	Registrar error y continuar	Error registrado, siguiente aceptada	PASS
TP-05	Rango invalido	Temperatura 999 C	Rechazar trama	Rechazada	PASS
TP-06	Desconexion	Cierre del socket	Registrar comunicacion perdida	Registrada	PASS
TP-07	Dispositivo ausente	Sin cliente TCP	Estado controlado <code>device_unavailable</code>	Reportado	PASS
TP-08	Persistencia	CSV/JSON/log en <code>output/</code>	Archivos legibles con metricas	Generados	PASS

Nota. Generado usando lenguaje de Markdown (MD).

7. Observabilidad y Reporte

Aunque en el video se explica en más detalle el código y su ejecución, aquí se presentará un resumen:

En la raíz del proyecto, se ejecuta el siguiente comando:

```
py -m hsf_telemetry.cli demo --messages 12 --interval 0.15
```

El cual sirve de demostración, de modo que ejecuta el firmware simulado, genera la telemetría, la envía por TCP hacia la aplicación de Python que valida y clasifica los datos, para posteriormente generar logs y archivos CSV/JSON. Se observan dos parámetros y sus valores en esta demostración: la cantidad de mensajes, y el intervalo para enviarlos de forma periódica en segundos. También existen otros dos comandos:

1. `py -m hsf_telemetry.cli simulator --messages 20 --interval 0.5`

El cual solo ejecuta el firmware simulado, de modo que genera la telemetría pero no se conecta con la aplicación de Python. En otras palabras, no valida ni guarda reportes, ya que es la capa de “dispositivo” que emite información.

2. `py -m hsf_telemetry.cli app --messages 12`

Ejecuta la parte de adquisición y procesamiento de datos, de modo que se conecta con el dispositivo y recibe los nuevos mensajes para posteriormente validarlos, clasificarlos y generar reportes.

Una vez generados y procesados los datos, se presentan las siguientes evidencias:

- Ejecución del primer comando:

Figura 6. Ejecución del proyecto.

```
PS C:\Users\quiqu\Desktop\Estudio\Code\IntSistemasInfoCorte> py -m hsf_telemetry.cli demo --messages 12 --interval 0.15
{
  "device_id": "SIM-001",
  "messages_received": 12,
  "valid_messages": 12,
  "normal_messages": 11,
  "anomalous_messages": 1,
  "errors": 0,
  "availability_percent": 100.0,
  "validity_percent": 100.0,
  "system": {
    "os": "Windows",
    "os_release": "10",
    "python": "3.11.2",
    "process_id": 19760,
    "process_count": 317,
    "memory_percent": 67.9,
    "disk_free_mb": 55151.85
  }
}
```

Nota. Captura de pantalla de la terminal Visual Studio Code.

Para este caso, se imprime por pantalla el resumen en formato JSON que quedará almacenado de forma local.

- ‘Logs’ en la ruta ‘outpuy/hsf.log’ después de algunas ejecuciones.

Figura 6. Archivo de ‘logs’ de ejemplo al ejecutar el proyecto.

```
output > hsf.log
1  2026-09-14 17:07:40,248 INFO integration_started port=58673
2  2026-09-14 17:07:40,250 INFO device_connected address=('127.0.0.1', 58674)
3  2026-09-14 17:07:40,270 INFO telemetry_accepted device=SIM-001 sequence=0 status=NORMAL
4  2026-09-14 17:07:40,291 INFO telemetry_accepted device=SIM-001 sequence=1 status=NORMAL
5  2026-09-14 17:07:40,311 INFO telemetry_accepted device=SIM-001 sequence=2 status=NORMAL
6  2026-09-14 17:07:40,331 INFO telemetry_accepted device=SIM-001 sequence=3 status=ANOMALO
7  2026-09-14 17:07:40,352 INFO telemetry_accepted device=SIM-001 sequence=4 status=NORMAL
8  2026-09-14 17:07:40,372 INFO telemetry_accepted device=SIM-001 sequence=5 status=NORMAL
9  2026-09-14 17:07:40,373 INFO integration_finished valid=6 errors=0
10 2026-09-14 17:09:10,169 INFO integration_started port=56847
11 2026-09-14 17:09:10,172 INFO device_connected address=('127.0.0.1', 56848)
12 2026-09-14 17:09:10,172 INFO telemetry_accepted device=SIM-001 sequence=0 status=NORMAL
13 2026-09-14 17:09:10,183 ERROR invalid_payload error=Expecting property name enclosed in double
    quotes: line 1 column 2 (char 1)
14 2026-09-14 17:09:10,194 INFO telemetry_accepted device=SIM-001 sequence=2 status=ANOMALO
15 2026-09-14 17:09:10,204 INFO telemetry_accepted device=SIM-001 sequence=3 status=NORMAL
16 2026-09-14 17:09:10,205 INFO integration_finished valid=3 errors=1
17 2026-09-22 17:11:07,503 INFO integration_started port=59834
18 2026-09-22 17:11:07,507 INFO device_connected address=('127.0.0.1', 59835)
19 2026-09-22 17:11:07,507 INFO telemetry_accepted device=SIM-001 sequence=0 status=NORMAL
20 2026-09-22 17:11:07,657 INFO telemetry_accepted device=SIM-001 sequence=1 status=NORMAL
21 2026-09-22 17:11:07,808 INFO telemetry_accepted device=SIM-001 sequence=2 status=NORMAL
22 2026-09-22 17:11:07,958 INFO telemetry_accepted device=SIM-001 sequence=3 status=NORMAL
23 2026-09-22 17:11:08,109 INFO telemetry_accepted device=SIM-001 sequence=4 status=NORMAL
24 2026-09-22 17:11:08,259 INFO telemetry_accepted device=SIM-001 sequence=5 status=NORMAL
25 2026-09-22 17:11:08,410 INFO telemetry_accepted device=SIM-001 sequence=6 status=ANOMALO
26 2026-09-22 17:11:08,561 INFO telemetry_accepted device=SIM-001 sequence=7 status=NORMAL
27 2026-09-22 17:11:08,711 INFO telemetry_accepted device=SIM-001 sequence=8 status=NORMAL
28 2026-09-22 17:11:08,862 INFO telemetry_accepted device=SIM-001 sequence=9 status=NORMAL
29 2026-09-22 17:11:09,012 INFO telemetry_accepted device=SIM-001 sequence=10 status=NORMAL
30 2026-09-22 17:11:09,162 INFO telemetry_accepted device=SIM-001 sequence=11 status=NORMAL
31 2026-09-22 17:11:09,165 INFO integration_finished valid=12 errors=0
32
```

Nota. Captura de pantalla de Visual Studio Code.

- Telemetría en la ruta 'output/telemetry.csv':

Figura 7. Telemetría de ejemplo al ejecutar el proyecto.

```
output > telemetry.csv > data
1 device_id,sequence,sent_at,received_at,temperature_c,vibration_g,voltage_v,status,processing_ms
2 SIM-001,0,2026-09-22T22:11:07.506380+00:00,2026-09-22T22:11:07.507380+00:00,23.295,0.46,5.03,NORMAL,0.034
3 SIM-001,1,2026-09-22T22:11:07.657680+00:00,2026-09-22T22:11:07.657680+00:00,22.29,0.614,4.973,NORMAL,0.061
4 SIM-001,2,2026-09-22T22:11:07.808421+00:00,2026-09-22T22:11:07.808421+00:00,22.232,0.603,4.907,NORMAL,0.037
5 SIM-001,3,2026-09-22T22:11:07.958850+00:00,2026-09-22T22:11:07.958850+00:00,23.735,0.428,4.918,NORMAL,0.061
6 SIM-001,4,2026-09-22T22:11:08.109486+00:00,2026-09-22T22:11:08.109486+00:00,23.698,0.731,4.925,NORMAL,0.042
7 SIM-001,5,2026-09-22T22:11:08.259963+00:00,2026-09-22T22:11:08.259963+00:00,22.893,0.651,5.09,NORMAL,0.072
8 SIM-001,6,2026-09-22T22:11:08.410628+00:00,2026-09-22T22:11:08.410628+00:00,95.0,0.559,5.095,ANOMALO,0.053
9 SIM-001,7,2026-09-22T22:11:08.561066+00:00,2026-09-22T22:11:08.561066+00:00,22.186,0.743,4.958,NORMAL,0.046
10 SIM-001,8,2026-09-22T22:11:08.711513+00:00,2026-09-22T22:11:08.711513+00:00,22.577,0.447,4.962,NORMAL,0.039
11 SIM-001,9,2026-09-22T22:11:08.862140+00:00,2026-09-22T22:11:08.862140+00:00,25.265,0.472,5.016,NORMAL,0.034
12 SIM-001,10,2026-09-22T22:11:09.012700+00:00,2026-09-22T22:11:09.012700+00:00,24.556,0.549,5.01,NORMAL,0.04
13 SIM-001,11,2026-09-22T22:11:09.162865+00:00,2026-09-22T22:11:09.162865+00:00,22.251,0.424,4.941,NORMAL,0.067
14
```

Nota. Captura de pantalla de Visual Studio Code.

- Resumen de la ejecución más reciente en la ruta 'output/summary.json':

Figura 8. Archivo de resumen de ejemplo al ejecutar el proyecto.

```
output > {} summary.json > ...
1 {
2   "device_id": "SIM-001",
3   "messages_received": 12,
4   "valid_messages": 12,
5   "normal_messages": 11,
6   "anomalous_messages": 1,
7   "errors": 0,
8   "availability_percent": 100.0,
9   "validity_percent": 100.0,
10  "system": {
11    "os": "Windows",
12    "os_release": "10",
13    "python": "3.11.2",
14    "process_id": 19760,
15    "process_count": 317,
16    "memory_percent": 67.9,
17    "disk_free_mb": 55151.85
18  }
19 }
```

Nota. Captura de pantalla de Visual Studio Code.

8. Defensa Técnica

En adición a la presentación del proyecto realizada hasta el momento, se dará respuesta a algunas preguntas técnicas planteadas en la guía de aprendizaje:

- ¿Qué responsabilidad pertenece al hardware y cuál al firmware? ¿Dónde se produce realmente cada transformación de datos?

Respuesta: En base a lo expuesto la responsabilidad del hardware es producir y transportar señales físicas; el firmware se encarga de tomar muestras simuladas y enviarlas. La transformación de los valores simulados ocurre en el firmware, la transformación de estos datos en objetos ocurre en la aplicación de Python, la cual en su capa de procesamiento se encarga de transformar las mediciones en estados, y por último en la persistencia los objetos se transforman en CSV/JSON.

- ¿Qué servicios del sistema operativo utiliza la aplicación Python?

Respuesta: En base a lo expuesto, el SO proporciona sockets TCP/IP, gestión de procesos, administración de permisos, gestión de procesos y archivos, memoria física y virtual, 'logging' de forma nativa, y control de tiempo ('timestamps').

- ¿Qué ocurre si el sistema operativo revoca permisos sobre un recurso utilizado por la aplicación?

Respuesta: Como se ha mencionado, si el SO revoca permisos de salida, 'write_outputs' produce un 'persistence_error' en el log; el proceso conserva el resumen en memoria y termina controladamente.

- ¿Qué ventajas y limitaciones tiene usar Python frente a C/C++ o Rust en una integración con dispositivos?

Respuesta: Python permite prototipar rápido, tiene JSON, sockets, logging y pruebas accesibles, pero su rendimiento y determinismo son menores que C/C++ o Rust, y depende del intérprete.

- ¿Qué parte del sistema debería ejecutarse en firmware y cuál en software de aplicación? Justifique.

Respuesta: En este proyecto el firmware debería encargarse de la lectura de datos, identificar el dispositivo que recolecta los datos, numerar la secuencia, registrar los datos temporales, preparar la trama con el formato definido, enviar la información periódicamente, etc.; estas funciones se asignan al firmware debido a que es la capa más

cercana al dispositivo que requiere operar con menor latencia. Por otra parte, el software se encarga de la lógica de negocio, validación de datos, almacenamiento y análisis; la razón es que el software de aplicación tiene más flexibilidad, más capacidad de evolución y mejores herramientas para procesamiento, trazabilidad y observabilidad.

- ¿Cómo cambiaría la arquitectura si se conectaran 100 dispositivos simultáneamente?

Respuesta: Se reemplazaría el servidor de un cliente por un servicio concurrente 'asíncrono' o un broker MQTT (Message Queuing Telemetry Transport), con autenticación por dispositivo, colas ('queues'), control de desbordamiento de memoria, almacenamiento por lotes y métricas por fuente.

- ¿Qué riesgos de seguridad aparecen al recibir datos externos o ejecutar comandos del sistema?

Respuesta: Inyección de datos maliciosos sin validar, inyección de comandos, denegación de servicio (DoS) debido a saturación de los recursos, manipulación de registros, errores de permisos, entre otros.

- ¿Cómo garantizaría integridad y trazabilidad de los datos?

Respuesta: Primero se validan los datos desde la fuente; también se verifican los formatos, atributos, tipos de datos, rangos y secuencias; adicionalmente se registra información respecto a qué dispositivo genera los datos, el momento, el estado, y los posibles errores. En general consiste en monitorear todos los eventos registrando toda la información necesaria que permita saber qué pasa con cada procesamiento de datos.

- ¿Qué diferencias podrían presentarse entre Windows y Linux?

Respuesta: Windows y Linux difieren en rutas, permisos, nombres de procesos, servicios y comandos de administración.

- ¿Cómo diseñaría una estrategia de actualización del firmware sin afectar el servicio?

Respuesta: El firmware debe tener un control de versiones (usando Git y GitHub) que permita actualizar los sistemas de forma segura, verificando su integridad y teniendo la capacidad de hacer 'rollback' (restaurar copias de seguridad) en caso de errores.

- ¿Qué pruebas adicionales aplicaría antes de llevar el prototipo a producción?

Respuesta: Pruebas previas a producción como lo pueden ser carga con 100 dispositivos, desconexión prolongada, datos de tiempo incorrectos, mensajes duplicados/reordenados,

disco lleno, rotación de logs, actualización/rollback, recuperación eléctrica y análisis de seguridad.

- ¿Qué componente sería el principal punto de falla y cómo lo convertiría en un componente tolerante a fallos?

Respuesta: El principal punto de falla en este proyecto es el enlace de comunicación y adquisición de datos, es decir, la conexión TCP entre el dispositivo y la aplicación, esto debido a que si falla, la aplicación se queda sin datos lo cual puede alterar la integridad de la información a analizar. Para convertirlo en un componente tolerante a fallos se deben implementar mecanismos de reconexión, almacenamiento temporal de forma local, verificación de datos duplicados (para cuando se reestablezca la conexión), llevar un registro de los fallos, etc.

9. Referencias

Martínez. C. (2026) Aula virtual de Integración de Sistemas de Información. Universidad Santo Tomás de Colombia.

Tanenbaum, A. S., & Bos, H. (2015). Modern operating systems (4th ed.). Pearson.

Stallings, W. (2018). Operating systems: Internals and design principles (9th ed.). Pearson.

Kurose, J. F., & Ross, K. W. (2021). Computer networking: A top-down approach (8th ed.). Pearson.

Bass, L., Clements, P., & Kazman, R. (2012). Software architecture in practice (3rd ed.). Addison-Wesley Professional.

Greengard, S. (2015). The Internet of Things. MIT Press.

Python Software Foundation. (2026). Python documentation (v3.12). <https://docs.python.org/3/>

Microsoft. (2026). Visual Studio Code documentation. <https://code.visualstudio.com/docs>

Git Core Team. (2026). Git reference manual. <https://git-scm.com/docs>

GitHub. (2026). GitHub docs: Getting started with GitHub. <https://docs.github.com/>

Canva. (2026). Canva design platform. <https://www.canva.com/>